3.2. Освобождение памяти Linux глубже

LRU/MGLRU · swap · PSI · cgroup v2 · OOM

рабочий набор

что реально нужно держать в RAM

освобождение

кого вытеснять и какой ценой

давление

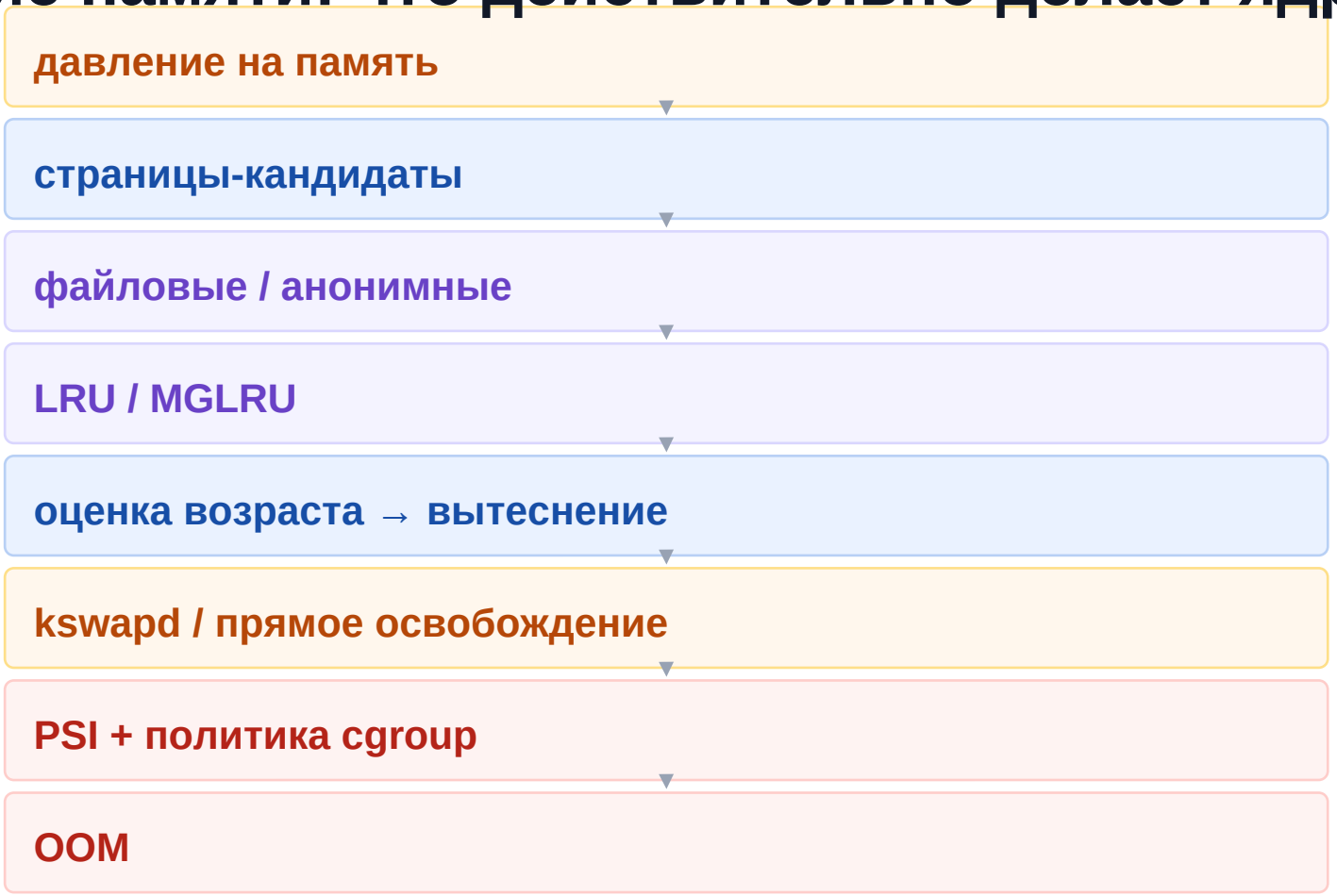
когда ядро замедляет приложение

лимиты

как cgroup доводит дефицит до OOM

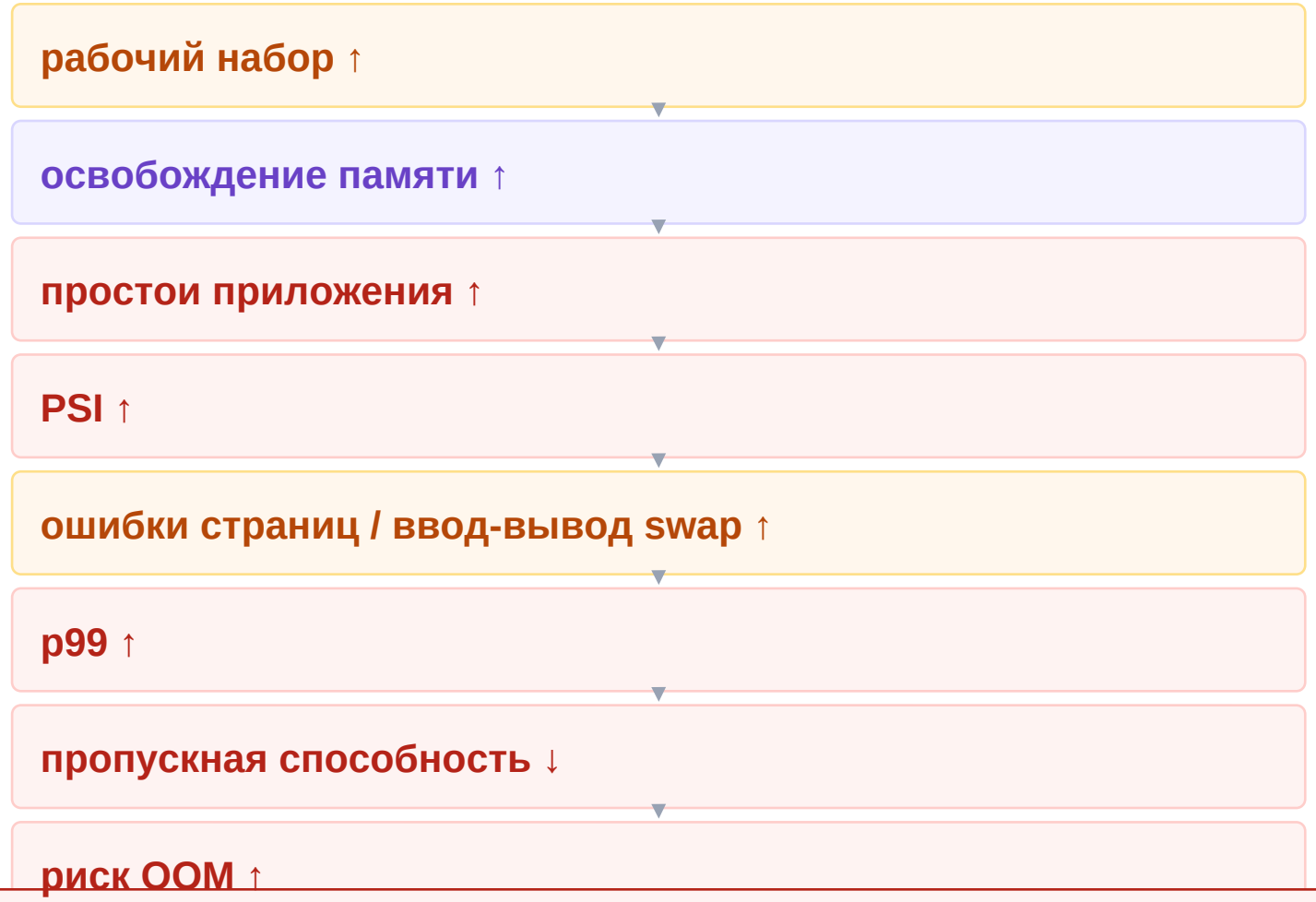
Linux постоянно освобождает память. OOM-killer вступает в конце цепочки, когда вернуть прогресс другими способами уже не получается.

Освобождение памяти: что действительно делает ядро



Вопрос ядра: какую страницу можно забрать сейчас с минимальным ущербом рабочему набору?

Под нагрузкой дефицит памяти увеличивает p99



OOMKilled - последний симптом. Деградация обычно начинается сильно раньше.

Ядро не может просто выбросить любые 4 КиБ

Можно выбросить

Чистая файловая страница: данные уже есть в файле. При необходимости ее можно перечитать.

Нельзя просто забыть

Анонимную или измененную файловую страницу придется где-то сохранить.

Перед вытеснением ядро проверяет происхождение и состояние страницы.

Чистая файловая страница — хороший кандидат

чистая файловая страница

вытеснить из RAM

повторное обращение

снова прочитать файл

Цена вытеснения возникнет при повторном обращении и чтении.

Измененную файловую страницу нужно сначала записать



При дефиците памяти скорость накопителя ограничивает освобождение страниц.

Анонимную память нельзя перечитать из файла



Без swap меньше способов вытеснять анонимные страницы.

Рабочий набор важнее размера адресного пространства

Процесс

Виртуальная память: 10 ГиБ

Активно используется: 2 GiB

Если 2 GiB помещаются

горячие страницы остаются в RAM; ядро освобождает холодные данным

Если рабочий набор > RAM

ядро начинает вытеснять данные, которые скоро снова понадобятся

Трэшинг возникает, когда рабочий набор не помещается в RAM, а не из-за большого VSS.

Откуда вообще берется идея LRU

Недавно использовали

Вероятность повторного обращения выше → страницу хочется защитить дольше.

Давно не использовали

Вероятность повторного обращения ниже → хороший кандидат для вытеснения.

Ядро не знает будущего и использует прошлую активность как эвристику.

LRU в Linux — не точная очередь обращений

Наивная модель

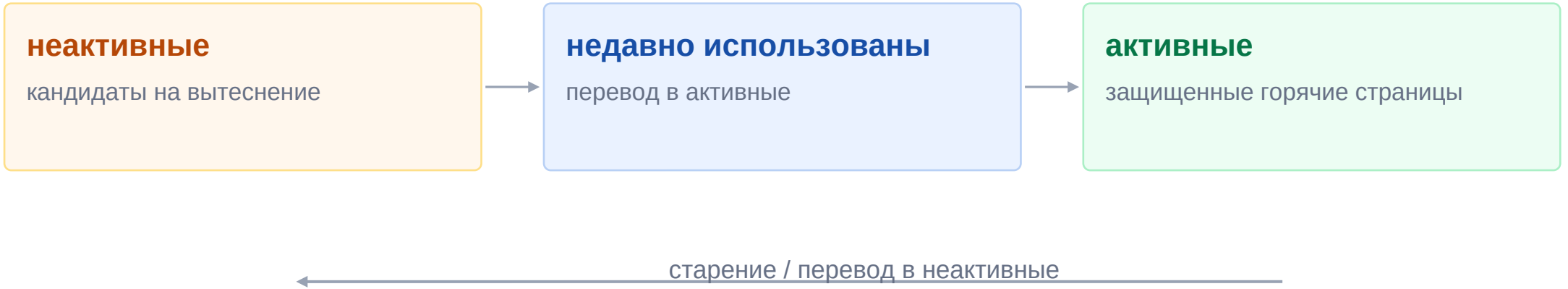
Каждое чтение или запись меняет позицию страницы в список по времени последнего обращения.

Практическая модель

Приближенная оценка возраста, группировка и признаки доступа. Дешевле, но неточно.

Точный LRU слишком дорог: учет обращений сам съедал бы производительность.

Классическая модель активных и неактивных страниц



Две корзины дают рабочую модель, но грубо различают возраст страниц.

Где двух корзин уже мало

горячую страницу приняли за холодную

↓

вытеснение

↓

быстрое повторное обращение

↓

повторная ошибка страницы

↓

лишняя ошибка страницы / I/O

Чем хуже оценен возраст, тем дороже ошибочное вытеснение.

MGLRU: возраст разбит на поколения



MGLRU точнее оценивает возраст, чем два списка. Нужны поддержка ядра и настройка.

MGLRU разделяет старение и вытеснение

Оценка возраста

Какие страницы еще горячие? К какому поколению они относятся?

Вытеснение

Какие старые страницы вытеснить при текущем дефиците?

Это два этапа одного цикла управления освобождением памяти.

Как ядро узнает, что страницу использовали

процесс обращается к странице

MMU / данные таблиц страниц

оценка возраста учитывает активность

страница остается молодой

MMU и таблицы страниц позволяют отслеживать обращения без проверки каждой инструкции.

Повторная ошибка: страницу вытеснили зря



Частые повторные ошибки означают: ядро вытесняет данные из рабочего набора.

MGLRU учитывает повторные ошибки страниц



Освобождение страниц — адаптивная политика, а не статическая сортировка.

swappiness - не процент использования swap

Плохая модель

vm.swappiness=60 → «держать swap заполненным на 60%».

Точная модель

Параметр влияет на выбор анонимной памяти
или файлового кеша при освобождении страниц.

swappiness меняет выбор страниц, но не задает процент заполнения swap.

Swap расширяет пространство решений

Без swap

Анонимную страницу с нужными данными нельзя просто выбросить.

Со swap

Содержимое можно записать в swap и освободить физическую страницу.

Цена этой возможности — ввод-вывод swap и дорогой возврат страниц.

Выгрузка в swap



Процесс работает дальше, но следующий доступ к странице будет дороже.

Загрузка из swap



Загрузка из swap требует I/O и может увеличить p99.

Swap полезен до тех пор, пока вытесняется холодное

Здоровый сценарий

Холодные анонимные страницы ушли в swap; RAM досталась горячему рабочему набору.

Трэшинг

Горячие страницы постоянно выгружают и возвращают. Ядро переносит данные вместо полезной работы.

SwapUsed > 0 сам по себе ничего не говорит о качестве работы системы.

zswap: тратим CPU, чтобы сократить I/O

страницу хотят выгрузить в swap

↓

сжать

↓

сохранить сжатой в RAM

↓

можно избежать немедленного I/O

zswap уменьшает I/O ценой CPU и все равно потребляет RAM.

Когда освобождение памяти замедляет запрос

Свободных страниц достаточно

обычное выделение памяти идет без заметного вытеснения

Запас падает

kswapd просыпается и пытается восстановить запас заранее

Фон не успевает

выделяющий память поток сам входит в прямое освобождение

Критический момент: ядро освобождает память прямо во время аллокации приложения.

kswapd: дорогую работу пытаются сделать заранее



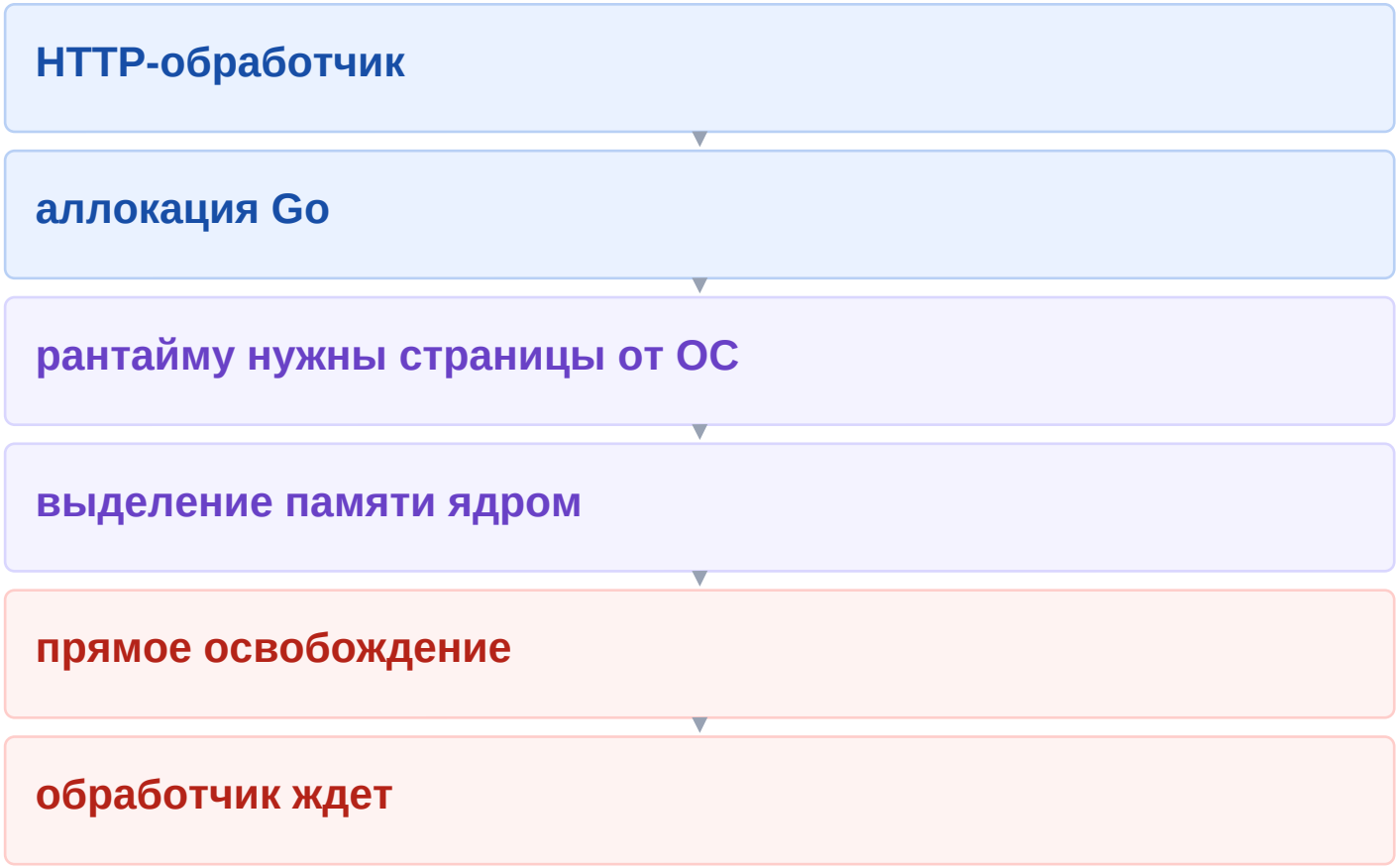
Задача kswapd — освободить память до того, как это замедлит потоки приложения.

Прямое освобождение: приложение платит само



Дефицит памяти увеличивает задержку даже при исправном GC Go.

Как прямое освобождение попадает в Go-обработчик



Снаружи это выглядит как «обычная аллокация вдруг стала медленной».

Освобождение памяти тратит CPU и I/O

- сканирование LRU/MGLRU и метаданных памяти;
- обход и обновление таблиц страниц;
- вытеснение чистых файловых страниц;
- запись измененных файловых страниц;
- выгрузка анонимных страниц в swap;
- учет памяти и ограничения cgroup.

Дефицит памяти способен увеличить CPU, обращения к диску и задержки.

Эффективность: сколько освободили при сканировании

Сканирование

pgscan-подобная идея:
просмотрели 1 000 000 страниц



Результат

pgsteal-подобная идея:
освободили только 5 000 страниц

Много ищем, мало освобождаем: CPU занят, дефицит памяти сохраняется.

Трэшинг: вытеснение разрушает рабочий набор

нужна A → вытесняем B

нужна B → вытесняем C

нужна C → вытесняем A

повторные ошибки / загрузки

полезная работа почти остановилась

Система перемещает страницы вместо полезной работы.

Почему при трэшинге CPU может быть загружен лишь на 50%

Что видим

p99 огромный, пропускная способность падает, CPU может оставаться ниже 100%.

Где время

задачи ждут освобождения памяти, ошибок страниц и I/O. Узкое место — не вычисления CPU.

Низкая загрузка CPU не исключает тяжелой проблемы с памятью.

PSI: сколько времени задачи теряют из-за нехватки памяти

```
/proc/pressure/memory

some avg10=... avg60=... avg300=... total=...
full avg10=... avg60=... avg300=... total=...
```

some

часть задач простаивает из-за дефицита памяти

full

все активные задачи одновременно простаивают

PSI показывает потерю времени из-за дефицита памяти, а не процент занятой RAM.

PSI some и full показывают разную тяжесть проблемы

some

Некоторые запросы/задачи застряли, другие продолжают работать. Система деградирует, но еще продвигается.

full

Все активные задачи одновременно ждут освобождения памяти. Это признак тяжелого трэшинга.

Продолжительный full - гораздо более тяжелый сигнал, чем просто some > 0.

95% занятой RAM может быть нормальнее, чем 75%

Система A

RAM занята на 95%
PSI ≈ 0
Освобождение памяти не мешает задачам.

Система B

RAM занята на 75%
PSI высокий
Приложение действительно простаивает.

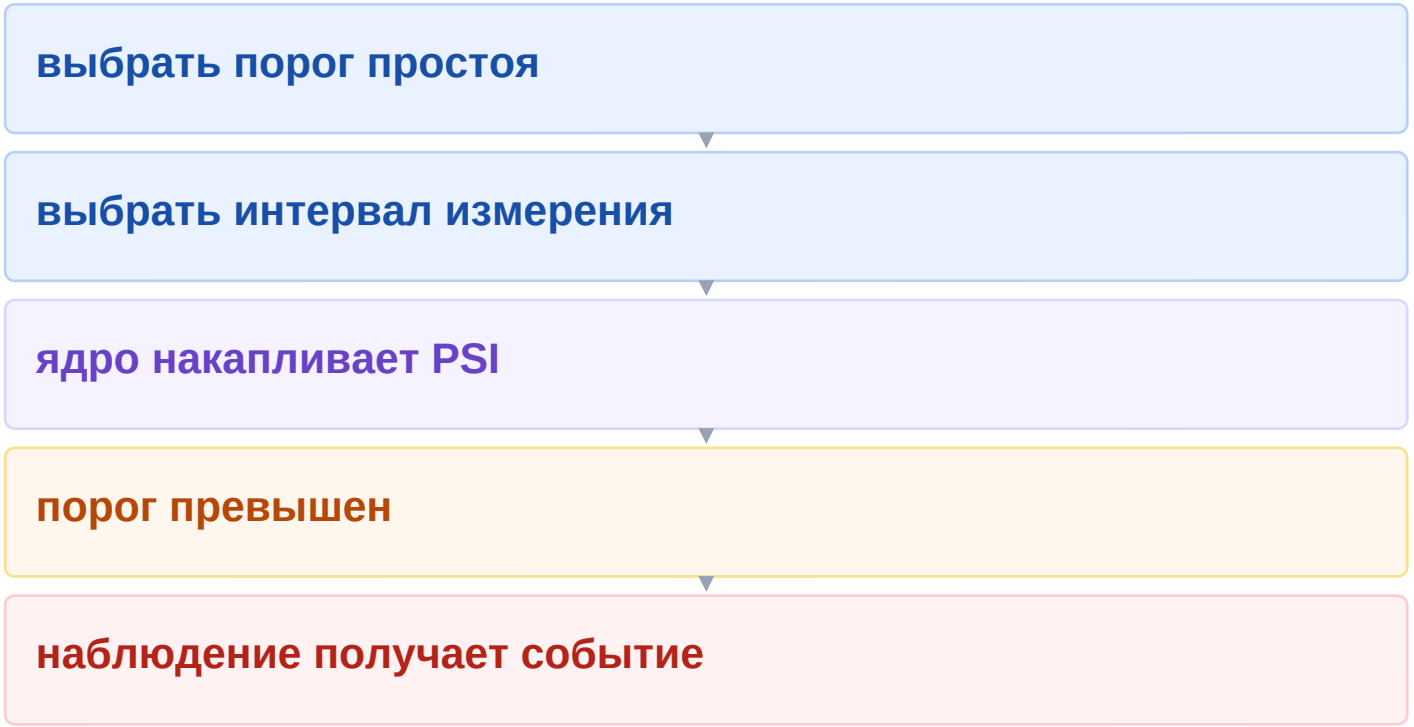
Процент занятой RAM — снимок состояния; PSI — цена нехватки памяти.

Как читать PSI во времени

- `avg10` - короткое окно, хорошо ловит свежую деградацию;
- `avg60` - средний тренд;
- `avg300` - длительное давление;
- `total` — суммарное время простоя;
- при сбое сопоставляем PSI с p95/p99 и пропускной способностью.

PSI полезен динамикой и корреляцией с SLO, а не одной точкой.

PSI можно использовать для оповещения о дефиците памяти



Оповещаем о простое задач, а не просто об использовании памяти > 90%.

cgroup v2: память контейнера шире хипа Go

Go

хип
стеки
метаданные рантайма

Процесс / ядро

mmap
сторонние аллокации
буферы сокетов
файловый кеш

Учет cgroup

anon
file
slab
TCP и прочие расходы ядра

`memory.current` и `HeapAlloc` отвечают на разные вопросы.

memory.current и memory.stat

memory.current

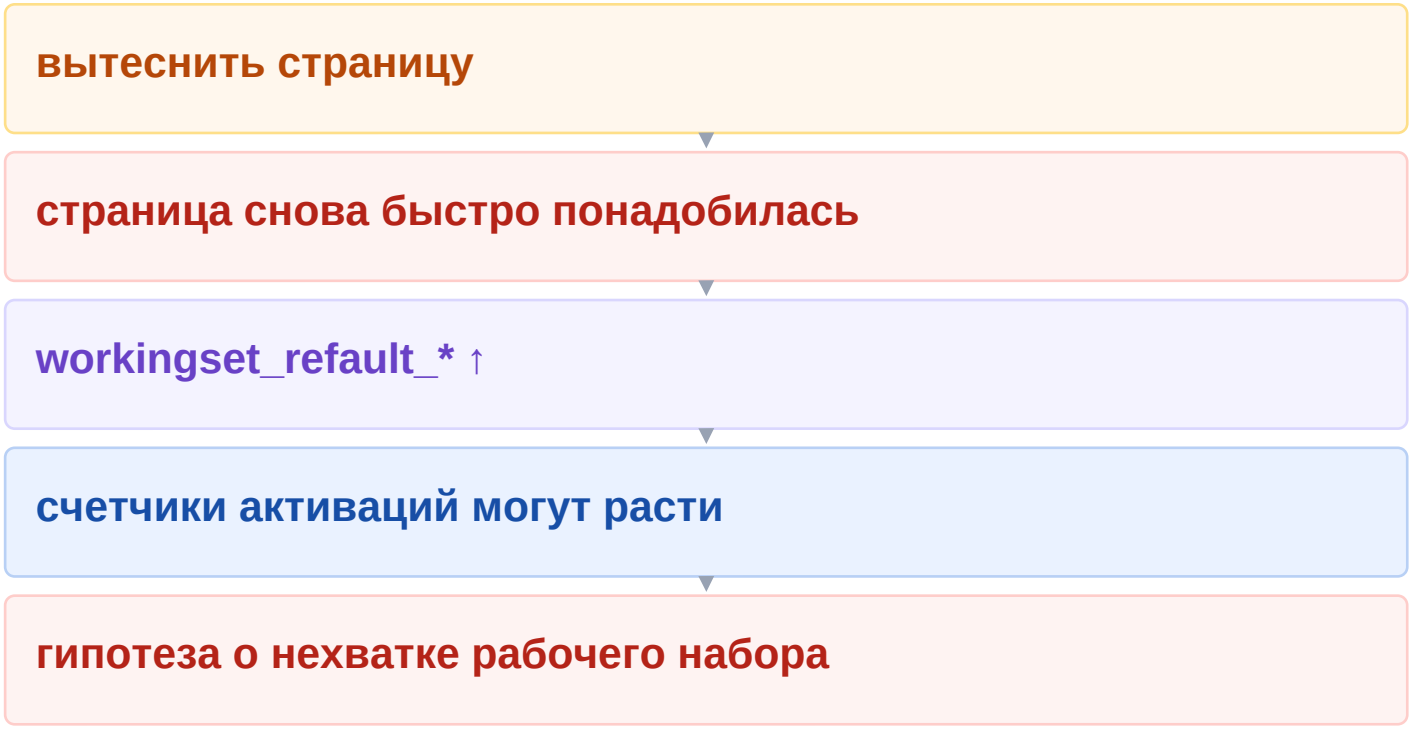
Сколько памяти cgroup и ее дочерних групп сейчас учитывает контроллер памяти.

memory.stat

Состав: anon, file, slab, ошибки страниц, счетчики рабочего набора и другие.

Первое дает размер, второе помогает построить гипотезу причины.

Счетчики рабочего набора показывают цену вытеснения



Счетчики повторных ошибок показывают: страницы выбрасывают и быстро возвращают.

pgfault и pgmajfault

pgfault

Все ошибки страниц: большинство может быть обычными ошибками без I/O (minor).

pgmajfault

Ошибки, требующие I/O. Важно сопоставить с p99 и дисковыми операциями или swap.

Один счетчик ошибок страниц без контекста ничего не доказывает.

memory.min и memory.low задают защиту от вытеснения

memory.min

Жесткая защита от вытеснения в допустимых пределах.
Если незащищенных страниц мало, OOM может прийти раньше.

memory.low

Мягкая защита: ядро сначала освобождает страницы менее защищенных cgroup, если возможно.

Обе настройки меняют приоритет вытеснения, но не резервируют физическую RAM.

Защита от вытеснения не резервирует RAM заранее

Плохая модель

memory.low=1G → ядро физически отложило 1 GiB только для этого контейнера.

Точная модель

Политика определяет, чью память освобождать позже, а чью защищать дольше.

Это приоритет освобождения, а не разделение физической памяти.

memory.high: управляемое замедление до OOM



Превышение `memory.high` само по себе не запускает OOM-killer.

memory.high особенно важен для задержек backend

```
memory.high = 900 MiB
memory.max  =   1 GiB
```

current превысил high

▼

освобождение / замедление

▼

p99 ↑

▼

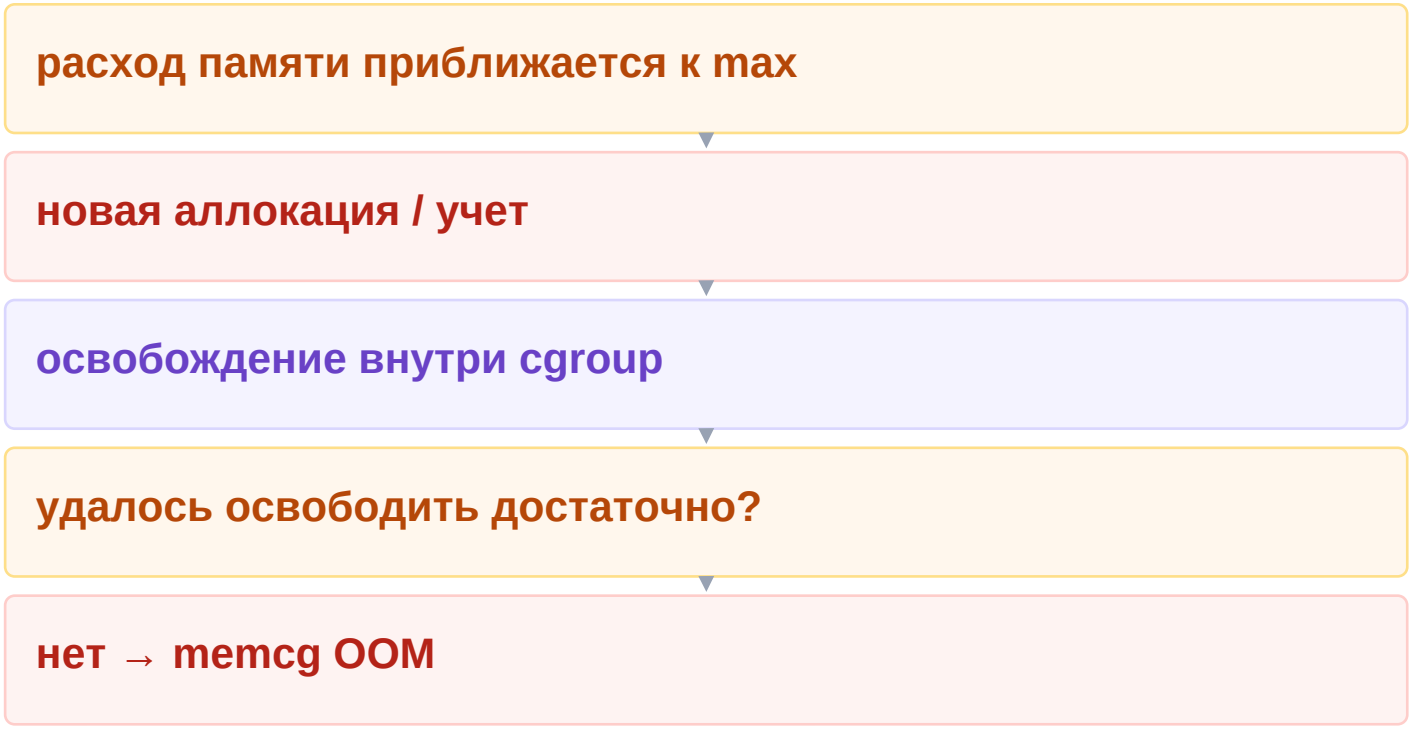
пропускная способность ↓

▼

OOMKilled = 0

Контейнер может быть серьезно деградирован задолго до OOM.

memory.max: жесткий лимит cgroup



memory.max — жесткий лимит: ядро сначала пытается освободить память.

Первый байт выше max не обязан мгновенно убить процесс

Что происходит сначала

Ядро пытается освободить память внутри cgroup и удалось ли получить прогресс.

Когда появляется OOM

Если освободить достаточно не удалось, выделение памяти завершается состоянием OOM.

До завершения процесса обычно уже заметно давление на память.

memory.events: хронология нехватки памяти

Счетчик	Что означает
high	частые превышения memory.high
max	операции доходят до жесткого лимита
oom	при выделении памяти возник OOM
oom_kill	OOM-killer реально убил процесс
oom_group_kill	завершена вся OOM-группа

`memory.events` помогает восстановить события до OOMKilled.

memory.events и memory.events.local

memory.events

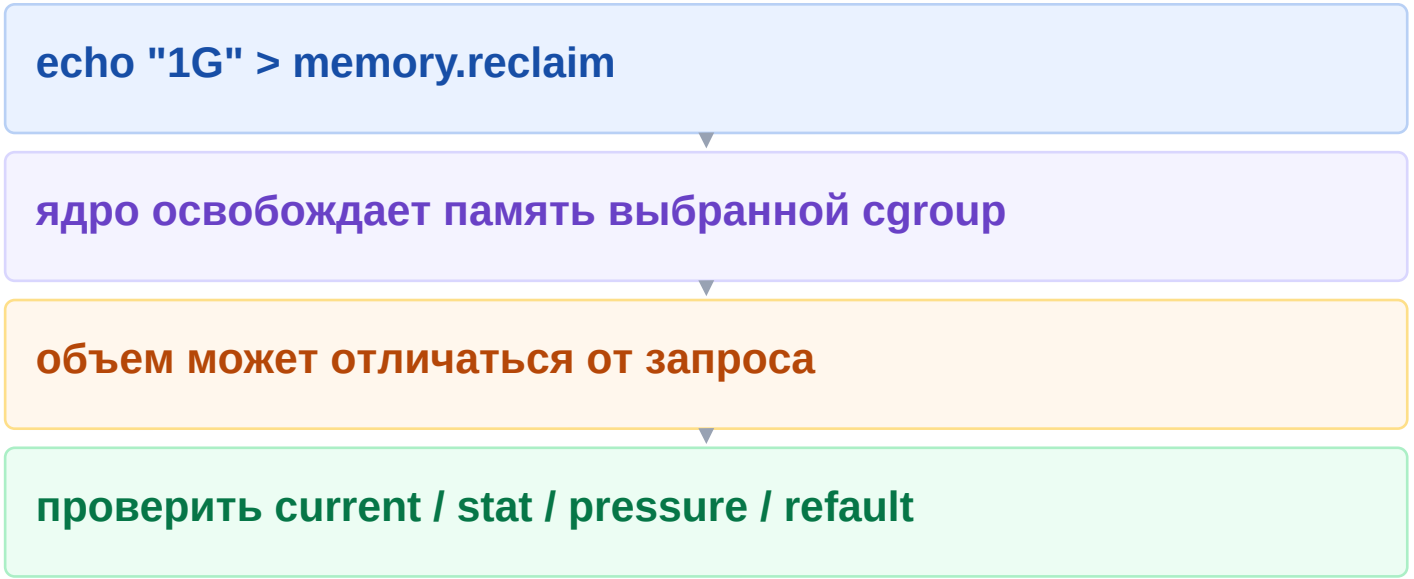
Может учитывать события дочерних cgroup.

memory.events.local

Показывает события только текущей cgroup.

В дереве cgroup это помогает точно найти источник проблемы.

memory.reclaim: ручной запрос освобождения памяти



Это полезный инструмент, но он не воспроизводит реальный дефицит памяти полностью.

memory.swap.current и memory.swap.max

swap.current

Сколько swap учитывает cgroup. Сам по себе этот объем не говорит о текущей деградации.

swap.max

Лимитирует, сколько анонимной памяти cgroup может отправить в swap.

Политика swap меняет доступные способы освобождения памяти.

memory.swap.max=0: цена отключения swap

Плюс

Нет дорогой загрузки из swap; меньше риск большой задержки во время запроса.

Минус

Холодные анонимные страницы нельзя выгрузить; ядро получает меньше вариантов, а OOM приходит раньше.

Отключение swap не является универсальной оптимизацией.

memory.pressure: PSI конкретного cgroup

Что измеряет

Сколько времени задачи этой cgroup простаивают из-за нехватки памяти.

Почему сильная метрика

Связывает действия ядра с реальной способностью приложения выполнять работу.

Для контейнера это более содержательный сигнал, чем один процент `memory.current / memory.max`.

memory.high + PSI показывают уже начавшуюся деградацию

current ≈ / > high

▼

memory.events high ↑

▼

PSI some ↑

▼

прямое освобождение замедляет приложение

▼

p99 ↑

Если full заметно растет, нехватка памяти уже серьезна.

Расход памяти и давление — разные показатели

Контейнер A

900 MiB / 1 GiB
PSI ≈ 0
приложение работает нормально

Контейнер B

700 MiB / 1 GiB
PSI высокий
приложение простаивает

Один процент занятой памяти не показывает, насколько система страдает.

OOM не означает, что свободных байтов ровно ноль

Наивная модель

Счетчик свободной памяти стал 0 → OOM.

Точная модель

Ядро не смогло выделить память при текущих ограничениях даже после попыток освобождения.

Важны состояние памяти, NUMA, cpuset, memcg и тип запроса.

Глобальный OOM: восстановить работу системы

системное выделение памяти невозможно

▼

освободить память не удалось

▼

глобальное состояние OOM

▼

выбрать процесс-жертву

▼

завершить процесс

▼

память освободилась → система работает

OOM-killer решает задачу выживания системы, а не архитектурного поиска виноватого.

memcg OOM: хост может быть почти пустым

на хосте есть свободная RAM

↓

cgroup достигла memory.max

↓

освобождение внутри cgroup не помогло

↓

memcg OOM

↓

жертву выбирают внутри cgroup

Изоляция cgroup сильнее наличия свободной RAM на ноде.

Почему под получает OOMKilled на свободной ноде

Нода

Свободная RAM: 40 ГиБ

Контейнер

memory.max: 1 GiB
расход ≈ 1 ГиБ

Результат

memcg OOM → процесс завершен → Kubernetes: OOMKilled

OOMKilled не требует исчерпания памяти всей ноды.

oom_score / oom_score_adj

/proc/<pid>/oom_score

Текущая оценка процесса при выборе жертвы.

/proc/<pid>/oom_score_adj

Меняет вероятность выбора процесса. Нехватка памяти при этом никуда не исчезает.

Защитили один процесс — ядро при необходимости выберет другой.

memory.oom.group сохраняет целостность приложения

приложение из нескольких процессов

гибель одного процесса оставит полусистему

memory.oom.group = 1

считать всю cgroup единым приложением

Групповой OOM полезен, когда частичный отказ хуже полного перезапуска.

Временная шкала до OOMKilled

- t0: `memory.current` растет;
- t1: фоновое освобождение усиливается;
- t2: PSI some растет;
- t3: прямое освобождение замедляет аллокации;
- t4: повторная ошибка страницы / swap / writeback растут;
- t5: p99 растет;
- t6: `memory.events high` может уже расти;
- t7: расход памяти приближается к max;
- t8: освободить достаточно памяти не удается;
- t9: memcg OOM; t10: процесс убит; t11: Kubernetes сообщает OOMKilled.

Следить лишь за OOMKilled — значит замечать проблему слишком поздно.

Go и Linux наблюдают разные уровни

Рантайм Go

/gc/heap/live · /gc/heap/goal · возврат хипа · CPU GC · помощь GC CPU · стеки горутин

Linux / cgroup

memory.current · stat · pressure · events · swap · ошибки страниц · освобождение памяти

Для расследования нужны обе картины одновременно.

Сценарий: хип Go мал, память cgroup почти исчерпана

- живой хип Go = 400 МиБ;
- memory.current = 1.8 GiB; memory.max = 2 GiB;
- проверяем файловый кеш / файловую память;
- нативные аллокации и cgo;
- анонимные mmap;
- стеки горутин;
- буферы сокетов и slab;
- общие / файловые отображения.

Профиль хипа показывает лишь часть общей памяти процесса.

Сценарий: Go GC нормальный, но p99 растет

Go

GC CPU ≈ 5%
STW короткий
хип стабилен

Linux

PSI памяти ↑
прямое освобождение ↑
ошибки с I/O ↑

Задержка может возникнуть в ядре при полностью исправном GC.

Два цикла обратной связи работают одновременно

Цикл рантайма Go

высокая частота аллокаций
↓
GC
↓
помощь GC от горутин
↓
CPU приложения уменьшается

Цикл памяти Linux

дефицит резидентной памяти
↓
освобождение памяти
↓
ошибки страниц / swap / простои
↓
задержка растет

Под нагрузкой две системы обратной связи способны усиливать друг друга.

Слишком маленький GOMEMLIMIT

GOMEMLIMIT ↓

▼

частота GC ↑

▼

CPU на помощь GC ↑

▼

CPU приложения ↓

▼

задержка ↑

▼

PSI Linux может оставаться низким

Сервис тормозит из-за GC Go, хотя ядро не испытывает нехватки памяти.

Слишком высокий GOMEMLIMIT

GOMEMLIMIT близок к memory.max

▼

хип занимает почти весь лимит

▼

другим расходам почти нет запаса

▼

освобождение внутри cgroup

▼

PSI ↑

▼

риск OOM ↑

Сэкономили CPU на GC, но усилили освобождение в ядре и риск OOM.

Скевенджер Go и освобождение памяти Linux — разные механизмы

Скевенджер Go

Рантайм знает: страницы хипа свободны → сообщает ОС, что физическую память можно вернуть.

Освобождение Linux

Ядру не хватает памяти, и оно ищет страницы, доступные для вытеснения из процессов и кешей.

Первое инициирует рантайм Go, второе — политика ядра Linux.

Нормальное освобождение памяти незаметно для backend

нехватка памяти немного выросла

kswapd проснулся

найдены холодные чистые страницы

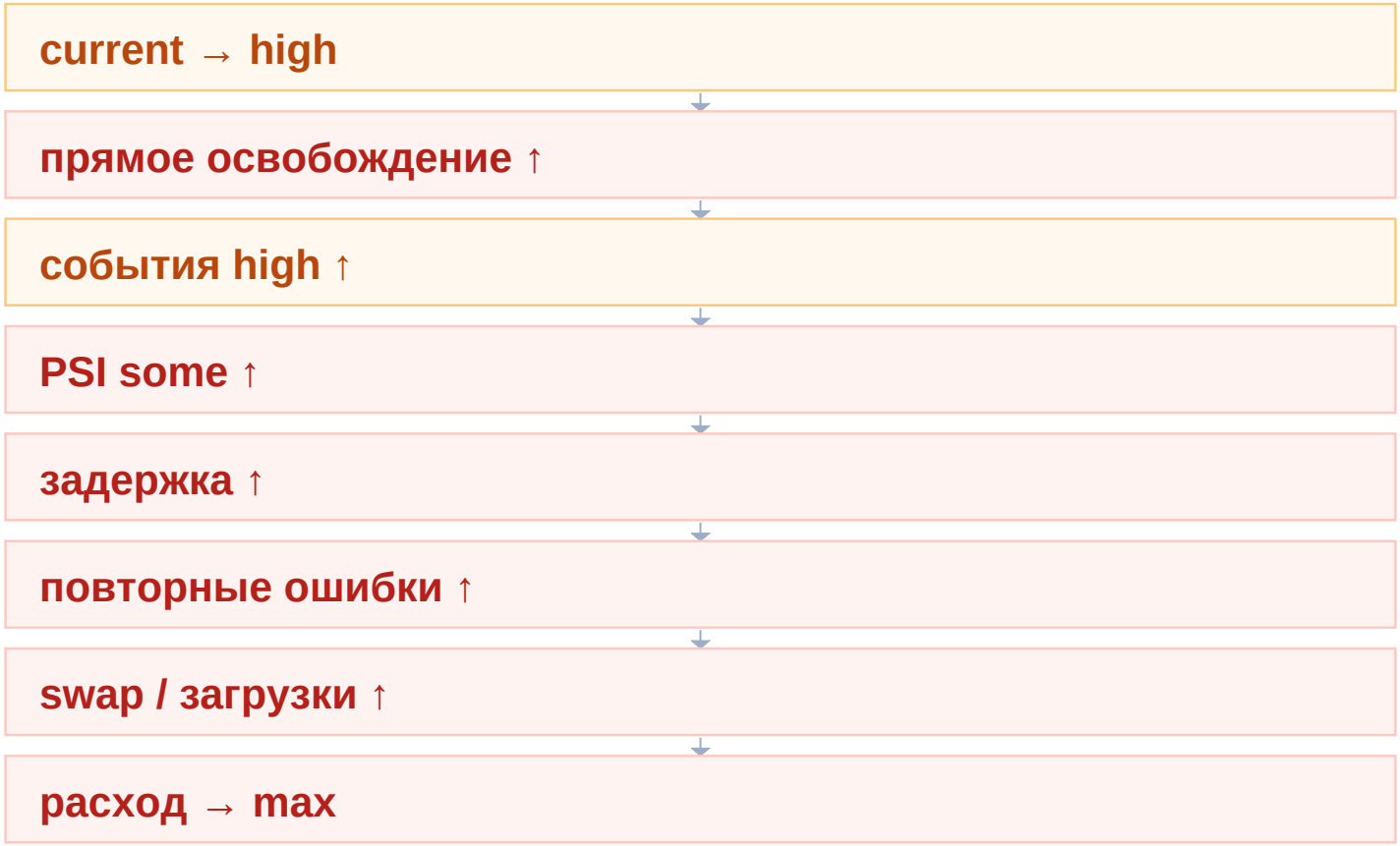
страницы освобождены

горячий рабочий набор сохранен

PSI ≈ 0

ОС постоянно освобождает страницы, обычно без заметных задержек приложения.

Деградация начинается до OOM



Процесс еще работает, но проблема уже развивается.

Признаки трэшинга

PSI full заметно вырос

▼

ошибки с I/O ↑

▼

повторные ошибки ↑

▼

swap I/O ↑

▼

пропускная способность падает

Ядро выбрасывает страницы, которые почти сразу снова нужны.

Системная диагностика: стартовый набор

- `free -h` — общая картина памяти;
- `vmstat 1` — swap, ожидающие задачи, I/O и динамика;
- `cat /proc/pressure/memory` — простои по всей системе;
- `cat /proc/meminfo` — состав памяти;
- `cat /proc/vmstat` — сканирование, вытеснение и ошибки страниц.

Отдельная метрика ничего не доказывает: важны динамика и связь с задержками.

vmstat: активность si/so важнее объема swap

Что смотреть

`si` / `so` — загрузка/выгрузка swap, ожидающие задачи, ожидание I/O и свободная память.

Сильная гипотеза

si/so постоянно активны + p99 плохой → swap участвует в горячем пути.

Холодные страницы могут лежать в swap месяцами без вреда.

/proc/vmstat: искать → освобождать → возвращать



Ключевой вопрос: сколько ищем, сколько забираем и сколько данных затем быстро возвращается?

Минимальная панель cgroup v2

Файл	Вопрос
memory.current	сколько памяти сейчас учитывает cgroup?
memory.stat	состав памяти и ошибки страниц?
memory.pressure	простаивают ли задачи?
memory.events(.local)	последовательность high/max/OOM?
memory.high / memory.max	порог замедления / жесткий лимит?
memory.swap.current / max	используется ли swap и насколько он разрешен?

Эти показатели позволяют различить исчерпание лимита и реальную деградацию.

Как расследовать OOM контейнера

OOMKilled? → memory.events oom_kill?

current близок к max?

события high росли раньше?

memory.pressure some/full?

memory.stat: anon/file/slab/faults/refaults

swap.current / активность swap?

параллельно: хип Go / CPU GC / помощь GC

Сначала восстановите историю дефицита, затем ищите его источник.

Практика: файловая и анонимная память

- Нагрузка A: прочитать большой файл → файловый кеш.
- Нагрузка B: анонимный буфер → коснуться каждой страницы.
- Создать дефицит памяти в изолированной среде.
- Сравнить `memory.stat`, swap, ошибки страниц и PSI.
- Показать, что эти типы страниц освобождаются по-разному.

Чистую файловую страницу можно выбросить. Анонимные данные нужно сохранить.

Практика: kswapd → прямое освобождение → p99

- Постепенно увеличивать рабочий набор в VM/cgroup.
- Измерять `/proc/vmstat`, `/proc/pressure/memory` и задержки.
- Найти начало усиленного фонового освобождения.
- Затем найти прямое освобождение и простои.
- Связать PSI и p99.

Цель: найти момент, когда освобождение памяти начинает задерживать поток приложения.

Практика: swap и PSI

- В отдельной VM включить swap.
- Создать анонимный рабочий набор больше RAM.
- Смотреть `vmstat si/so`, ошибки с I/O, PSI и задержки.
- Повторить без swap.
- Сравнить: OOM позже ↔ возможные большие задержки загрузки.

Swap дает больше способов освободить память, но меняет характер задержек.

Практика: memory.high

- Настроить `memory.high < memory.max`.
- Поднять рабочий набор выше high, но ниже max.
- Смотреть `memory.events high`, `memory.pressure`, p95/p99.
- Убедиться: процесс жив, OOM нет, задержки уже растут.

Так можно показать управляемое замедление еще до OOM.

Практика: memory.max и OOM

- Работать только в отдельной cgroup / контейнере.
- Довести расход памяти до жесткого лимита.
- Смотреть `memory.events`: max → oom → oom_kill.
- Зафиксировать PSI и задержки до завершения процесса.
- Сопоставить с Kubernetes OOMKilled.

Цель: увидеть нехватку памяти → неудачное освобождение → memcg OOM.

Практика: memory.reclaim и повторная ошибка страницы

- `echo "256M" > memory.reclaim`;
- до и после измерить current/stat/pressure и ошибки страниц;
- затем снова активно обратиться к данным;
- посмотреть, что быстро возвращается и какой ценой.

Принудительное освобождение полезно в опыте, но не равно реальному дефициту памяти.

Практика: Go и cgroup как два цикла управления

- Go: живой хип / цель, CPU GC и помощь GC;
- Linux: current, pressure, events, stat, swap;
- Backend: RPS, p50, p95, p99;
- менять `GOMEMLIMIT`, `memory.high`, `memory.max`;
- определить, где доминирует GC Go, а где освобождение Linux.

Практика связывает рантайм Go, память Linux и SLO.

Четыре плохих диагноза

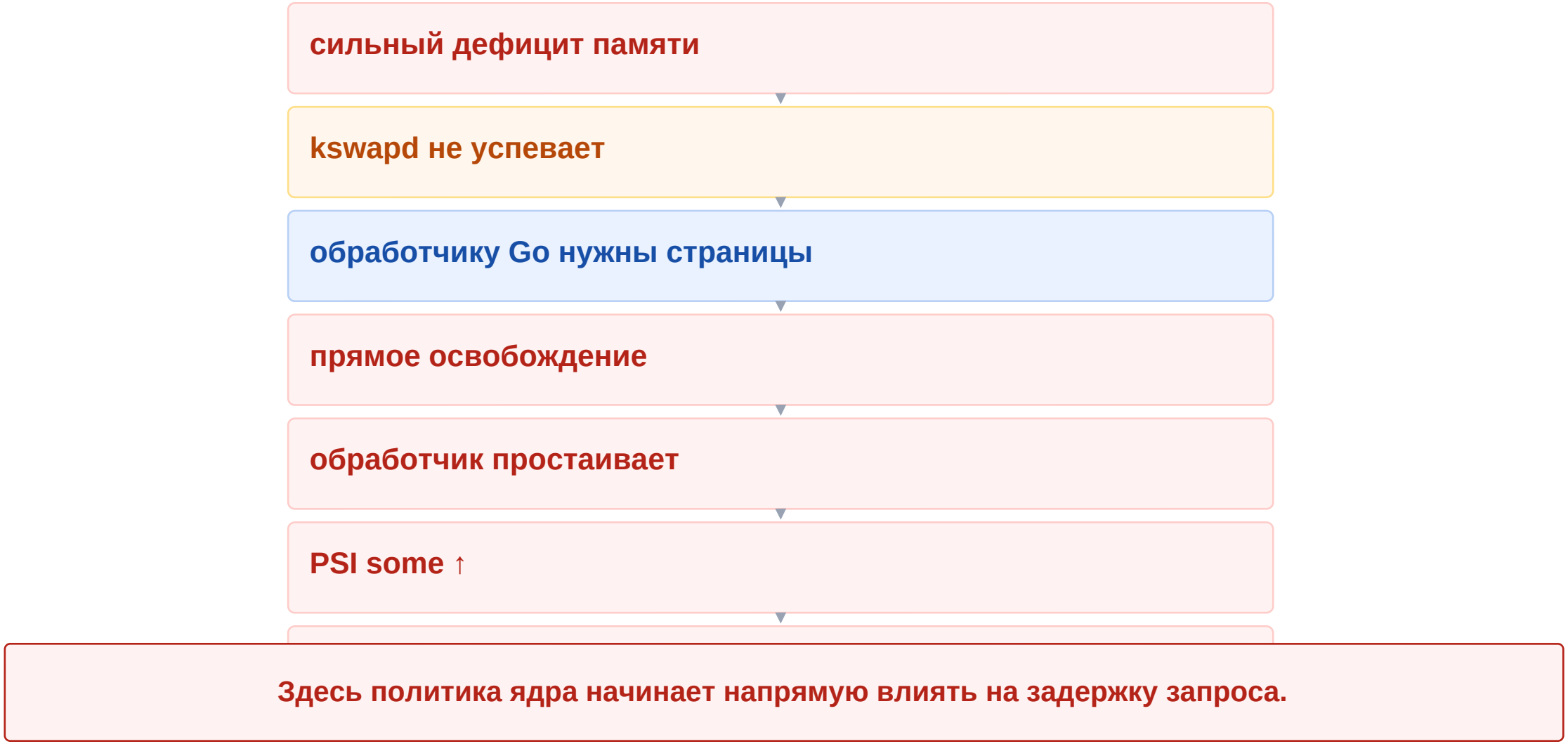
Упрощение	Что смотреть вместо этого
«Файловый кеш украл память»	повторные ошибки, запись страниц, анонимная память, PSI
«SwapUsed > 0 - сервер умирает»	si/so, ошибки с I/O, PSI, задержки
«PSI some > 0 - авария»	величина, длительность, динамика, связь с SLO
«MGLRU решит нехватку RAM»	рабочий набор и доступная RAM

Умная политика вытеснения не отменяет физический предел RAM.

Сквозная цепочка: здоровый освобождение памяти



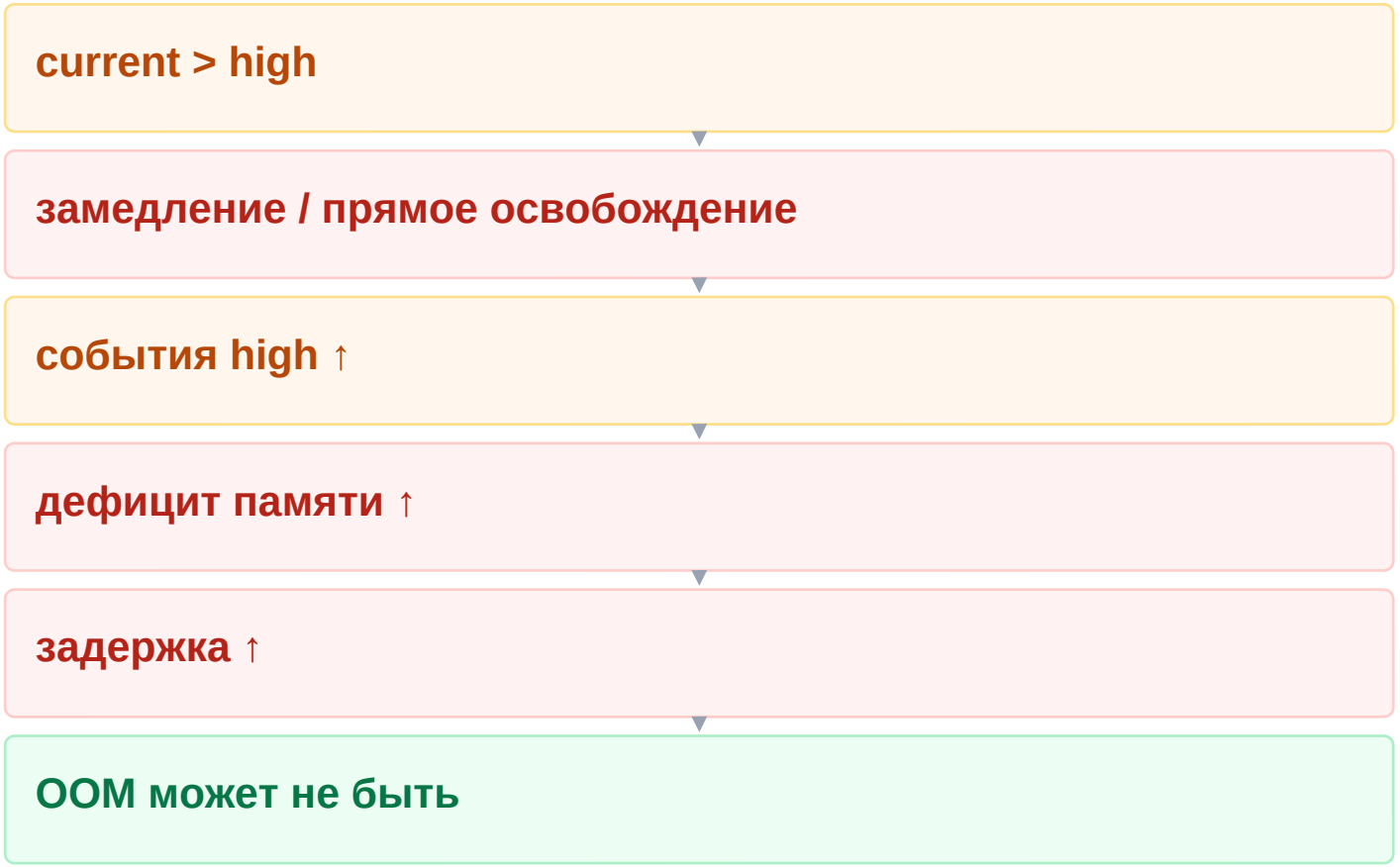
Сквозная цепочка: освобождение памяти в запросе



Сквозная цепочка: трэшинг



Сквозная цепочка: memory.high



memory.high создает фазу замедления до жесткого отказа.

Сквозная цепочка: memcg OOM



Порочный круг деградации

RPS ↑

↓

частота аллокаций Go ↑

↓

резидентная память ↑

↓

превышение memory.high

↓

прямое освобождение / PSI ↑

↓

задержка запросов ↑

↓

запросы в работе ↑

↓

живых данных ↑ → дефицит сильнее

При перегрузке рост незавершенных запросов увеличивает расход памяти.

Как разорвать порочный круг

Приложение

ограничить параллелизм
сократить удержание памяти
уменьшить частоту аллокаций
сократить рабочий набор

Рантайм Go

реалистичный GOMEMLIMIT
следить за помощью GC
оставлять запас памяти

Контейнер

адекватный memory.max
memory.high
осознанная политика swap

Linux

измерять PSI / освобождение
не создавать
вредную политику защиты
страниц

Единого регулятора нет: дефицит формируется на нескольких уровнях.

Какой инструмент отвечает на какой вопрос

Вопрос	Инструмент
Есть ли простои системы?	/proc/pressure/memory
Простаивают ли задачи cgroup?	memory.pressure
Сколько памяти учтено?	memory.current
Из чего состоит расход?	memory.stat
Достигнуты high/max/OOM?	memory.events
Есть ли swap I/O?	vmstat / sar
Как ведет себя хип Go?	runtime/metrics + pprof

Диагностика начинается с вопроса к системе, а не с любимой метрики.

Что устойчиво, а что зависит от версии ядра

Устойчивая модель

нехватка → кандидаты → рабочий набор
оценка → старение → вытеснение →
повторные ошибки → источники данных

Публичные интерфейсы

/proc/pressure/*
cgroup v2 память.*
/proc/<pid>/oom_score_adj

Детали реализации

внутренности MGLRU
число поколений
объем сканирования
пороги свободной памяти
правила выбора жертвы OOM

Разделяйте общую модель, публичные интерфейсы и детали конкретного ядра.

Что важно запомнить

- Ядро освобождает страницы с минимальным ущербом рабочему набору.
- Чистые файловые страницы дешевле вытеснять; анонимные данные надо сохранить.
- LRU приблизительно оценивает возраст; MGLRU учитывает поколения и повторные ошибки.
- kswapd работает в фоне; прямое освобождение замедляет поток приложения.
- Swap расширяет возможности вытеснения, но возврат страниц увеличивает p99.
- PSI измеряет простои: some — часть задач, full — все активные задачи.
- memory.high вызывает замедление; memory.max — жесткий лимит и возможный OOM.
- OOMKilled — конец цепочки; дефицит памяти и p99 обычно растут раньше.
- GC Go и освобождение Linux — разные циклы, способные усиливать друг друга.

Главная цепочка: доступная RAM → рабочий набор → вытеснение → простои → SLO → OOM.

Финальная модель освобождения памяти Linux

ПРИЛОЖЕНИЕ ▸ рабочий набор ▸ дефицит памяти ▸ LRU / MGLRU ▸ освобождение ▸ PSI / cgroup ▸ OOM

Файловая

чистая → выбросить
изменена → записать

Выполнение

kswapd
↕
прямое освобождение

Анонимная

swap / zswap
или меньше способов вытеснения

Аллокации Go / живой хип → GOMEMLIMIT → RAM → memory.current → освобождение Linux → PSI → задержка запросов → риск OOM

Linux постоянно управляет рабочим набором. OOM-killer включается лишь тогда, когда обычные способы освободить память уже не помогают.

Официальные источники лекции

- Linux Kernel Documentation - Multi-Gen LRU: docs.kernel.org/mm/multigen_lru.html
- Control Group v2: docs.kernel.org/admin-guide/cgroup-v2.html
- PSI: docs.kernel.org/accounting/psi.html
- память Management Concepts: docs.kernel.org/admin-guide/mm/concepts.html
- /proc/sys/vm: docs.kernel.org/admin-guide/sysctl/vm.html
- zswap: docs.kernel.org/admin-guide/mm/zswap.html
- Physical память: docs.kernel.org/mm/physical_memory.html

Детали MGLRU и политики вытеснения зависят от версии и настроек ядра.